

TYPING TUTOR IN C

SOFTWARE DESIGN SPECIFICATIONS

Guide: Dr. Krishnapriya

Group:

Athul Babu K (NIE22CS019)

Sobin Seby (NIE22CS047)

Sojo Jenson (NIE22CS048)

OBJECTIVES

1. To present the design considerations and system architecture of the typing tutor software.
2. To explain the development methods and tools used in the project.
3. To showcase the user interface design and system components.
4. To highlight the key features and functionalities of the application.

ABSTRACT OF THE PROJECT

- **What is CMD TypeSwift?**

- A terminal-based typing tutor for programmers.
- Helps improve typing speed and accuracy while learning programming concepts.

- **Key Features:**

- Real-time feedback, error tracking, speed analysis, and ability to lessons.

- **Target Audience:**

- Programmers and anyone learning computer science who prefer command-line interfaces.

PROPOSED TOOLS AND SELECTION RATIONALE

- **Tools:**

- **GCC Compiler:** For compiling C code.
- **Git:** For collaboration and version control.
- **GNOME Terminal, xterm, uxterm:** For terminal compatibility.
- **Bash Scripting:** For installation and uninstallation scripts.

- **Selection Rationale:**

- GCC: Robust and widely supported.
- Git: Industry standard for version control and collaboration.
- Terminal Emulators: Widespread adoption among programmers.
- Bash: For simple and effective scripting.

SYSTEM OVERVIEW

- **Functionality:**

- Typing lessons with real-time feedback.
- Speed tracking (CPM and WPM).
- Error tracking and session results.
- Ability to add custom lessons via 'myown.txt'.

- **Design Considerations:**

- Lightweight and efficient.
- Terminal-based for Unix-like systems.
- Focus on user feedback and customization.

DESIGN CONSIDERATIONS

- **Assumptions and Dependencies:**

- Unix-like OS(Linux, macOS)
- Compatible terminal emulator (GNOME Terminal, xterm, uxterm).
- GCC Compiler for C code.

- **General Constraints:**

- Minimal hardware requirements.
- Unix-like environment and command-line familiarity.

DEVELOPMENT METHODS

- **Modular Programming:**
 - Application divided into modules (UI, lesson manager, error tracker, etc.).
- **Test-Driven Development (TDD):**
 - Unit tests for each module to ensure reliability.

SYSTEM ARCHITECTURE

- **High-Level Overview:**

- **User Interface (UI):** Handles user interaction and feedback.
- **Lesson Manager:** Manages lesson loading and selection.
- **Error Tracker:** Tracks error in real-time.
- **Speed Analyzer:** Calculates typing speed.
- **Sesson Logger:** Records session details to be displayed later.

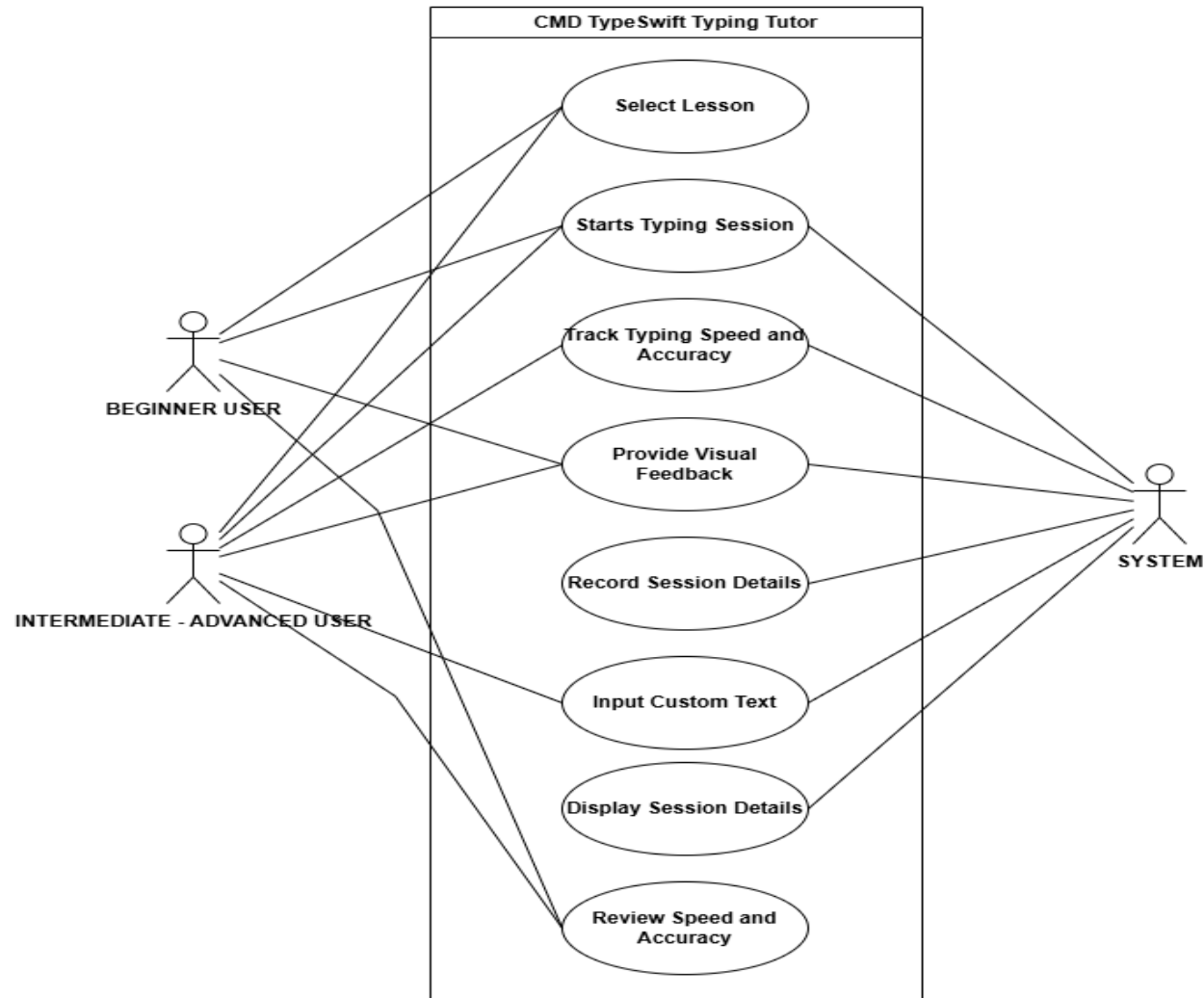
- **Component Interaction:**

- UI interacts with lesson manager, error tracker, and speed analyzer.
- Session logger records details during each session.

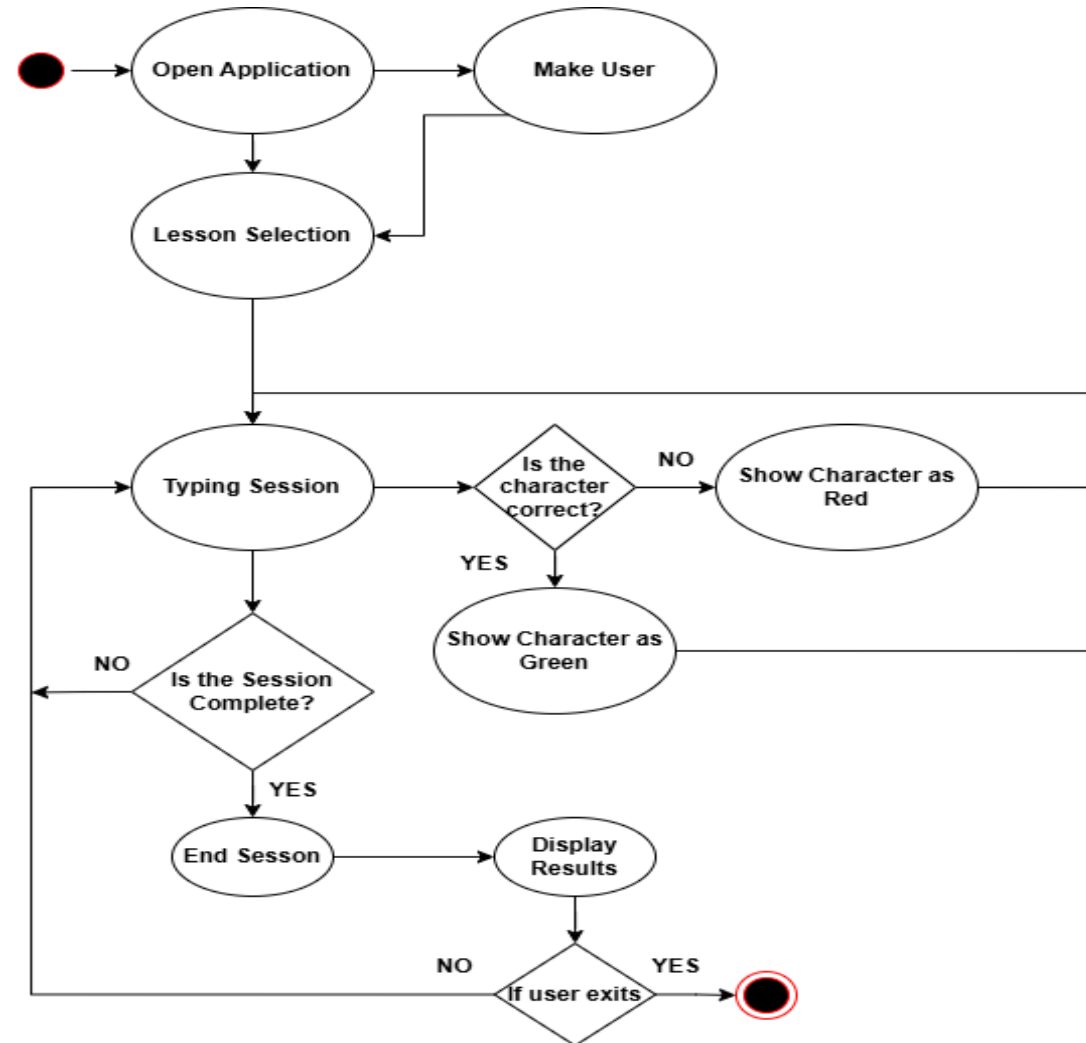
DETAILED SYSTEM DESIGN

- **User Interface (UI):**
 - Terminal-based with colour-coded feedback.
 - Commands like *cmd typswt --help* and *cmd typswt mkuser*.
- **Lesson Manager:**
 - Loads lessons from predefined or user-defined files.
- **Error Tracker:**
 - Monitors and logs errors in real-time.
- **Speed Analyzer:**
 - Calculated Characters Per Minute (CPM) and Words Per Minute (WPM).
- **Session Logger:**
 - Records session details

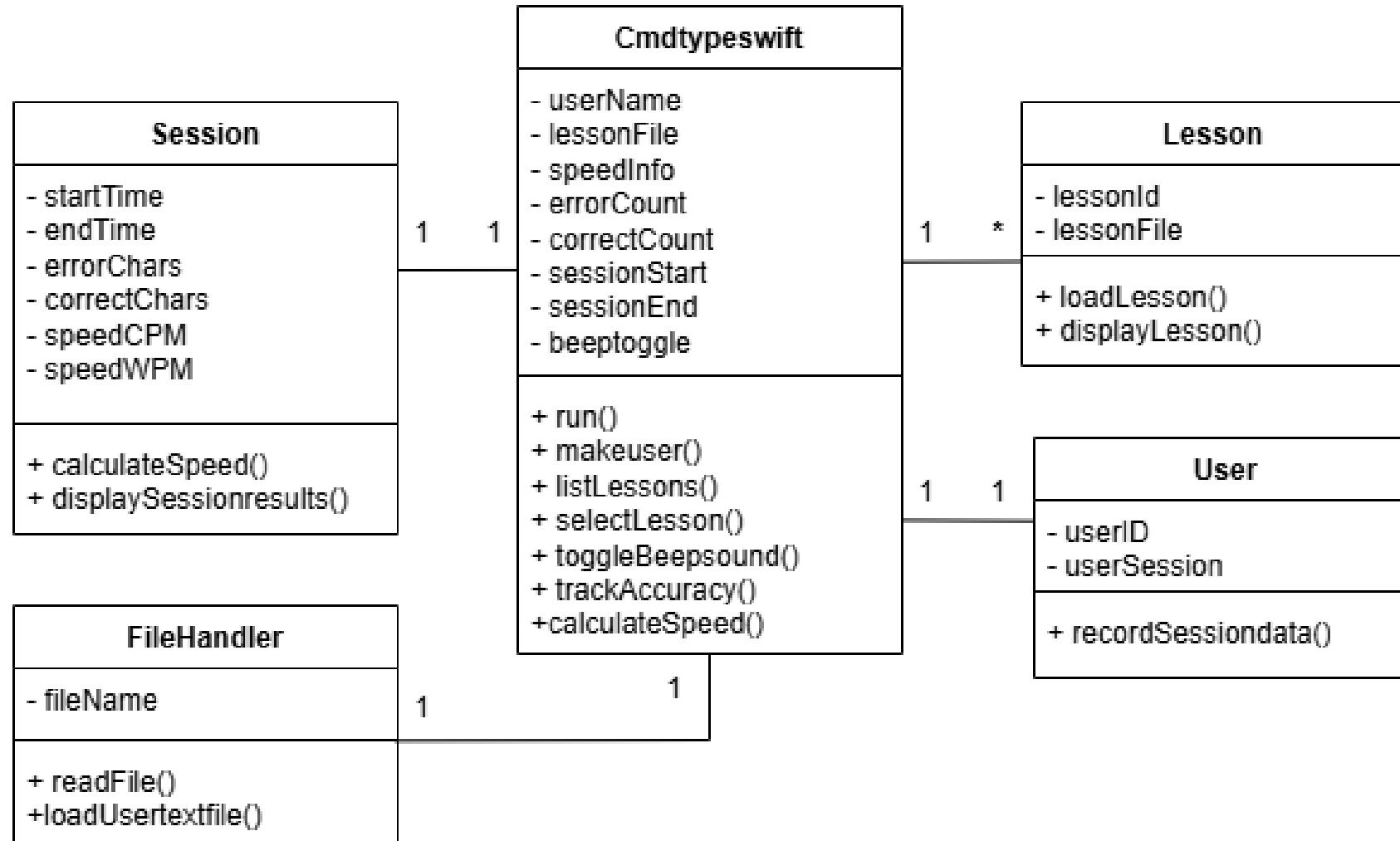
UML DIAGRAMS: Use Case Diagram



UML DIAGRAMS: Activity Diagram



UML DIAGRAMS: Class Diagram



INTERFACES (UI) AND USER ROLES

- **User Interface Screens:**

- **Main menu:** Displays available commands and options.
- **Lesson Selection:** Allows users to select a lesson.
- **Typing Session:** Displays lesson text with real-time feedback.
- **Session Summary:** Shows errors, speed, and time.

- **User Roles:**

- **New Users:** Can start with basic lessons.
- **Experienced Users:** Can add custom lessons.
- **Administrators:** Can manage lesson files and system settings.

UI MODELS

```

+*****+ +*****+ +*****+ +*****+
|*****| |*****| |*****| |*****|
**      **      **      **      **
**      **      **      **      **
**      **      **      **      **
**      **      **      **      **
|*****| |*  **  *| |*****|
+*****+ +*      *+ +*****+

```

CMD TYPIST HELP MENU

Cmdtypist Commands:

0) select "lesson-number":

where lesson-number is a valid cmdtypist lesson number. This command is run at the cmdtypist prompt.
Exampe: Enter command >>select 8, select is the command and 8 is a valid lesson number.

1) cmdtypist mkuser "username":

Creates a new user and stores username in the user_speed text file. Running this command erases current name in user_speed file. This command must be run by first time cmdtypist users. This command is mandatory once for every first time user. Running other commands for the first time without running this command will show and error message mandating this command. Running this command when a user name already exist deletes the current user to the new user name specified as second argument. If you want to preserve saved data in the user_speed file, then copy and paste in a different directory or rename it.

2) cmdtypist ls:

List available lessons for typing. Whenever the lessons are listed, the command select "lesson number" has to be followed to select lesson.

UI MODELS

```
LESSONS, use command <select 'lesson number' to make a choice:
1: Beginner lessons
2: Shell programming
3: Linux commands
4: Cmd commands
5: Learning numbers
6: Noslac's notes in computing
7: History of Cameroon
8: Getting acquainted to symbols
9: Coding in c
10: Coding in java
11: Coding in python
12: Coding in c++
13: Random word typing
14: Capital letter training
15: Mixed lessons
:: Writing manually to the user info file distorts display!!!
Enter command >>select 1

j j j j j k k k k l l l ; ; ; f f f d d d s s s a a a j k l ; f d s a f j k l-
j j j j j k k k k l l l ; ; ; f f f d d d s s s a a a j j j ; f a a a a a a [ ]
```

CONCLUSION

- **Recap of Objectives:**
 - Presented the design considerations and system architecture.
 - Explained the development methods and tools used.
 - Showcased the user interface design and system components.
 - Highlighted the key features and functionalities.

THANK YOU